The OreSat C3 Scheduling Architecture & Guardrails

Author: Chris Kane-Pardy **Date:** February 2026

Introduction & Purpose

The OreSat C3 Flight Computer operates in Low Earth Orbit. If a software bug causes the processor to freeze, a queue to flood, or the system to run out of memory, there is no human operator to press the reset button. An unhandled system lockup results in a "soft-brick," potentially ending the mission.

This document serves as the guidelines for the Scheduling Architecture and Resource Guardrails proposed for deployment on the C3. It explains the integration of Linux Control Groups (cgroups v2), `systemd` service management, and kernel Real-Time (RT) scheduling.

Goal of this Document: To ensure future engineers understand *why* the system is configured this way, exactly *what* every parameter does, and the risks of changing them without proper analysis.

System Architecture & The OLAF Concurrency Model

The C3 utilizes a single-core ARM Cortex-A8 processor running Debian Linux.

The Independent Processes (PID Level)

At the operating system level, the satellite's core logic is split into distinct processes managed by `systemd` :

- **`oresat-c3`**: Python application containing all flight logic.
- **`oresat-watchdog`**: A tiny, compiled Rust binary that protects the hardware.
- **`dbus-daemon` & `gpsd`**: Background system services.

The OLAF Threading Model (Inside `oresat-c3`)

The OreSat Linux App Framework (OLAF) dictates how the `oresat-c3` Python process is structured. Crucially, OLAF utilizes `threading.Thread` for service concurrency, not `multiprocessing`. This means all critical flight services share the same Process ID (PID), the same pool of memory, and the same Global Interpreter Lock (GIL).

The critical threads running inside this single process include:

- **The "Petter" (`__main__.py`)**: Wakes up every 1 second; sends a UDP "PET" packet to the Watchdog every 10 seconds.
- **StateService**: Manages the mission state machine (Boot -> Deploy -> Science).
- **NodeManager**: Routes traffic to CAN bus hardware (Star Tracker, Battery).
- **RadiosService**: Polls UDP sockets for ground station commands.
- **BeaconService**: Calculates CRC32 checksums and transmits 1Hz heartbeat pulses.
- **EdlService**: Parses uplink commands and handles CFDP (File Transfers) to the SD card.

Vulnerability Analysis

An extensive audit of the architecture above revealed three potential vulnerabilities.

Vulnerability A: The Global Interpreter Lock (GIL) Starvation

Because all OLAF services run as Python threads, they are subject to the **Global Interpreter Lock (GIL)**. Only one thread can execute Python bytecode at a time.

- **The Mechanism:** If `BeaconService` performs a heavy CPU calculation, or `NodeManager` gets flooded with CAN bus traffic, they hold the GIL.
- **The Catastrophic Risk:** The "Petter" thread shares this GIL. If the Petter thread is starved of CPU time for more than 30 seconds, the hardware Watchdog timer expires, and the satellite physically cuts power and reboots. This is a **false-positive reboot** caused entirely by software thread contention.

Vulnerability B: Infinite Queues & Memory Exhaustion (OOM)

The application threads communicate internally using unbounded Python Queues (`queue.SimpleQueue`). For example, the `RadiosService` reads data from the antenna and instantly dumps it into the `recv_queue` for the `EdlService` to process.

- **The Mechanism:** If the ground station floods the uplink (intentionally or via a glitch), `RadiosService` may fill the queue much faster than `EdlService` can process it.
- **The Catastrophic Risk:** If the queue grows infinitely in RAM. Eventually, the `oresat-c3` process consumes all 1GB of the board's physical memory. This might cause the entire Linux Kernel to panic, freezing the OS and permanently bricking the satellite.

Vulnerability C: Synchronous File I/O

The `EdlService` handles downloading large files (like images) from the ground or saving them to the 16GB eMMC flash storage.

- **The Mechanism:** Flash storage performs hardware-level "wear leveling" and garbage collection. During these cycles, write operations (`vfs.write`) can stall for hundreds of milliseconds.
- **The Catastrophic Risk:** Because the file write is synchronous, the stalled thread would block the Python interpreter. This leads straight back to Vulnerability A: the GIL is locked, the Petter thread cannot run, and the satellite reboots.

Data Flow & Inter-Process Communication (IPC)

To understand how to safely restrict these processes, we mapped the system's Inter-Process Communication (IPC). The system relies primarily on UDP sockets for safety-critical paths.

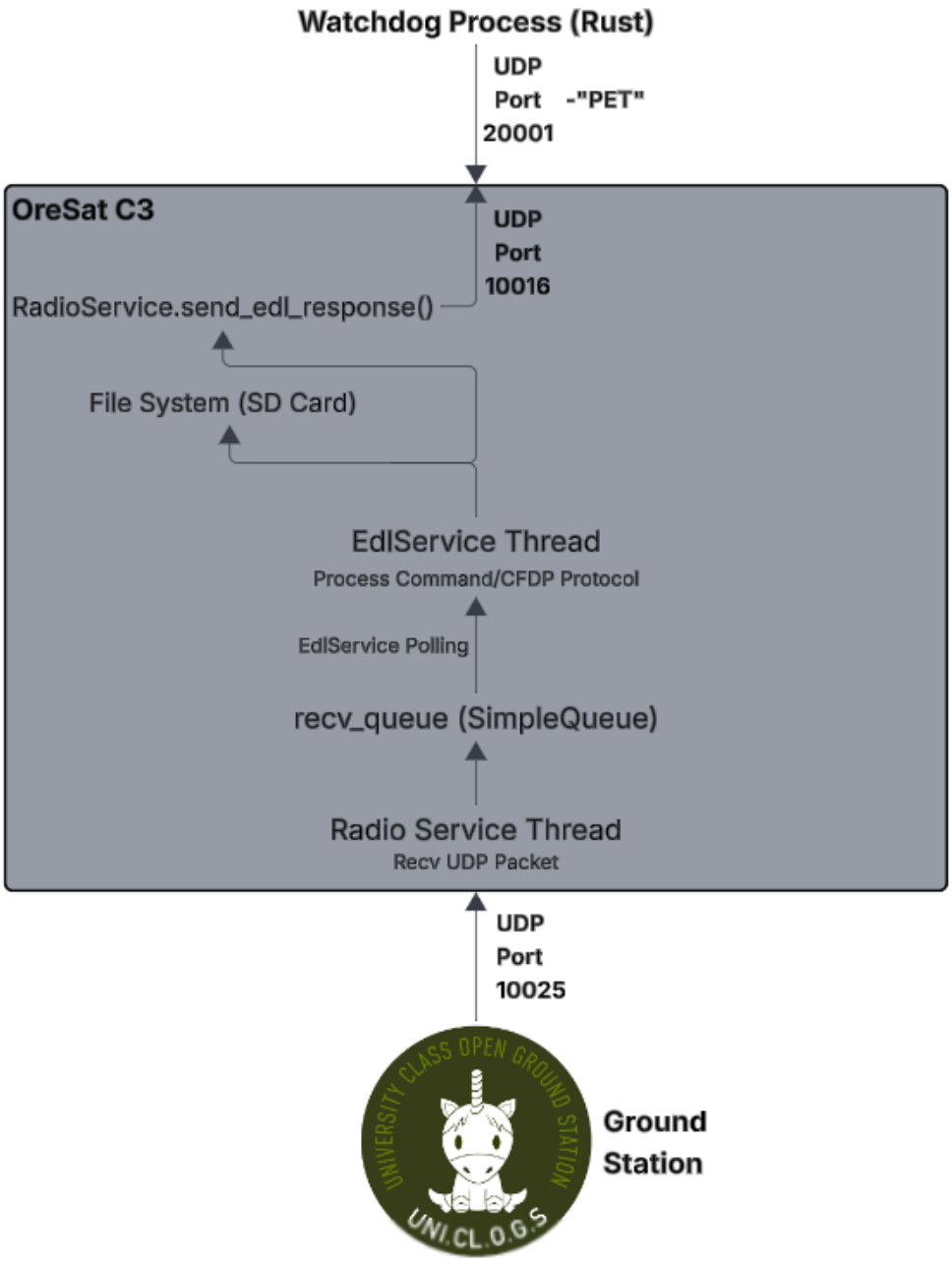
External & Internal IPC Matrix

Networking topology:

Direction	Source	Destination	Protocol	Function
Input	Ground / Sim	`RadiosService`	UDP `0.0.0.0:10025`	Telecommand / File Uplink
Output	`RadiosService`	Ground / Sim	UDP `10015`	Beacon (Heartbeat)
Output	`RadiosService`	Ground / Sim	UDP `10016`	File Downlink / Responses
Internal	`__main__.py`	`oresat-watchdog`	UDP `localhost:20001`	Safety "Pet"

Architectural Note (Why UDP for the Watchdog?): The internal safety "Pet" uses UDP for resilience; UDP is connectionless. If the Watchdog process crashes and is restarted by `systemd`, the Python application does not need to handle a `BrokenPipeError` or attempt to reconnect. It blindly continues firing UDP packets, and the Watchdog immediately picks them up the millisecond it re-binds to Port 20001.

The System Data Flow Map



The Dual-Tier Resource Architecture

To mitigate GIL starvation, unbounded queues, and I/O blocks, we abandoned application-level code fixes. Instead, we implemented an **Operating System Resource Cage** managed by the Linux Kernel via `systemd`.

The system is divided into two tiers of execution.

Tier 1: Safety Critical (The Watchdog)

- **Binary:** `oresat-watchdog` (Compiled Rust)
- **Role:** Listens on `localhost:20001`. If it receives a "PET" packet, it toggles the physical GPIO pin. If 30 seconds pass without a packet, it lets the hardware reboot the board.
- **Protection:** It is removed from standard scheduling entirely and granted Real-Time Priority over the Linux kernel. No matter what the Python app is doing, the OS will pause it to let the Watchdog run.

Tier 2: Mission Critical (The Flight Software)

- **Binary:** `oresat-c3` (Python)
- **Role:** The brains of the satellite (Radios, Deployment, Telemetry).
- **Protection (The Cage):** It is placed in a cgroup with a **high CPU weight** (to protect it from background OS tasks like logging) but a **strict Memory limit**. If the queue floods (Vulnerability B), the kernel kills only the Python process, leaving the Watchdog and OS perfectly stable to restart the software.

Systemd & Kernel Parameter Glossary

If you are maintaining or modifying the `.service` files in `/etc/systemd/system/`, you must understand these exact parameters. They are not arbitrary; they are the literal implementation of the architecture described above.

The Watchdog Parameters (`oresat-watchdog.service`)

- **`CPUSchedulingPolicy=rr`**
  - **What it is:** Changes the Linux scheduler policy for this PID from `SCHED_OTHER` (Standard completely fair scheduling) to `SCHED_RR` (Real-Time Round Robin).
  - **What it does:** Instructs the kernel that this process is exempt from "fairness."
  - **Why we use it:** It guarantees the watchdog will instantly preempt any infinite loop happening in user space.
- **`CPUSchedulingPriority=99`**
  - **What it is:** The highest possible Real-Time priority in the Linux Kernel (Range 1-99).
  - **What it does:** Places this daemon at the absolute front of the execution queue.
  - **Why we use it:** Ensures the Watchdog beats out any other Real-Time processes that future developers might add to the system.

- **`MemoryMax=32M`**
    - **What it is:** A cgroups v2 absolute RAM limit.
    - **Why we use it:** Rust is highly memory efficient. If this binary hits 32 Megabytes, it has suffered a memory leak and must be reaped before it harms the overarching system.

The Flight Software Parameters (`oresat-c3.service`)

- **`CPUWeight=500`**
    - **What it is:** A cgroups v2 proportional CPU allocation (Default is 100).
    - **What it does:** When the CPU is at 100% capacity, a weight of 500 guarantees `oresat-c3` gets 5x more CPU time than background tasks.
    - **Why we use it:** Protects telemetry intervals (jitter) and radio comms from being delayed by random OS background noise.
- **`MemoryMax=256M`**
    - **What it is:** The trigger for the Linux Out-Of-Memory (OOM) Killer. It will instantly send a `SIGKILL` to the Python process if it requests more than 256MB of RAM.
    - **Why 256MB:** The board has 1024MB of RAM. We reserve ~128MB for the OS. This leaves roughly ~600MB completely untouched. **This is intentional.** Linux automatically uses unused RAM as the "Kernel Page Cache", a write buffer for the slow eMMC flash.
- **`MemorySwapMax=0`**
    - **What it is:** Explicitly forbids the Linux kernel from "swapping" memory pages to the flash storage when RAM is full.
    - **What it does:** When `oresat-c3` hits 256MB, it forces the kernel to instantly kill it.
    - **Why we use it:** With slow flash, swapping is fatal. If swap is enabled, instead of killing the broken process, the CPU will spend time moving memory to and from the disk (`iowait`), which can lead to the entire OS freezing. We *want* the app to crash cleanly.

The Kernel Override (`/etc/sysctl_overrides.conf`)

- **`kernel.sched_rt_runtime_us=-1`**
    - **What it is:** A global sysctl override.
    - **What it does:** Disables the Linux kernel's default Real-Time throttling mechanism.
    - **Why we use it:** By default, Linux reserves 5% of CPU time for non-RT tasks to prevent a bad program from completely locking up the computer. However, in modern Cgroup v2 environments, enforcing this 5% quota often causes the kernel to reject the `CPUSchedulingPolicy=rr` request entirely

(`status=214/SETSCHEDULER` error). Setting this to `-1` forces the kernel to accept our Priority 99 Watchdog.

The Systemd Boot Sequence

Because the C3 Flight Software and the Watchdog communicate over UDP (a connectionless protocol), temporal boot ordering is critical. If the Flight Software boots *before* the Watchdog opens its port, the "PET" packets will be fired into the void, dropped by the kernel, and the hardware will reset the board 30 seconds later.

Systemd strictly enforces this topology:

```
[ Hardware Power On ]
         ↓
[ basic.target ] (OS Base Initialization)
         ↓
[ network.target ] (Loopback / Network Interfaces Up)
         ↓
         → (Watchdog needs localhost UDP port bound)
         ↓
[ oresat-watchdog.service ] (Priority 99 Real-Time Safety Net)
         |
         → (Requires=oresat-watchdog.service)
            (After=oresat-watchdog.service)
         ↓
[ oresat-c3.service ] (Python Flight Software)
```

`Requires=` vs `After=`: Requires means that if the Watchdog binary is deleted or corrupted, the Python software will *refuse* to start. After means the OS waits until the Watchdog reaches the active (running) state before it even attempts to execute the Python script.

Known Architectural Trade-Offs

During hardware validation, we discovered that implementing these strict Cgroup cages introduces two specific behavioral artifacts, which are intentional trade-offs, not bugs.

8.1 The Cgroup Bulkhead Paradox (CPU/Memory Stress)

When the Python application is subjected to extreme load and begins to approach its 256MB boundary, the Linux kernel must step in to perform **"Direct Reclaim."** The kernel briefly pauses

the Python interpreter, scans the 256MB cage, and frees up unused memory pages so the application doesn't hit the `MemoryMax` kill switch.

- **The Symptom:** This Direct Reclaim pause can stall the Python GIL for up to **~2.35 seconds** (according to data pulled from hardware testing).
- **The Logging Trap:** Because `CPUWeight=500` makes Python run 5x faster than the logging daemon `(systemd-journald)`, under 100% stress, Python generates log messages faster than Journald can read them. The Linux logging pipe fills up, forcing Python to block synchronously until the logger catches up.
- **The Conclusion:** The legacy architecture (no cgroups) actually produced telemetry under no load. However, while loaded, the latency delay produced equivalent performance to the optimized architecture with real-time scheduling. This optimized architecture **trades perfect determinism for guaranteed survivability** against catastrophic failures.

8.2 The Page Cache Flush (I/O Stress)

By capping the Flight Software at 256MB, we intentionally reserve ~600MB of RAM for the Linux Kernel Page Cache. This absorbs the slow write latency of the embedded eMMC flash. However, this buffer has a mathematical limit.

- **The Symptom:** If the satellite attempts a massive, sustained write operation, the 600MB Page Cache will eventually reach absolute capacity. When this happens, the kernel is forced to halt buffering and execute a massive, synchronous hardware flush (`fsync`) directly to the storage bus.
- **The Impact:** During this flush, the system experiences a temporary, bounded latency spike. In our physical hardware tests, this manifested as a **~658 ms** telemetry delay lasting for exactly 10 seconds before instantly returning to a perfectly flat, low-jitter baseline.
- **The Conclusion:** This is safe. The ~658 ms penalty is well within the 30-second Watchdog timeout constraint. It proves the architecture successfully absorbs catastrophic I/O bottlenecks and recovers autonomously without spiraling into a permanent `iowait` system freeze.

Future Developer Guide (Maintenance & Scaling)

If you are a future contributor taking over this project, follow these strict rules to prevent fatal regressions.

Rule 1: Never Manually Set Priorities in Code

Do not use `os.nice()` or `threading.setprofile()` inside the Python application. The Linux kernel (via systemd and cgroups) is the absolute source of truth. Application-level priorities conflict with Cgroup quotas and cause undefined, unpredictable kernel behavior.

Rule 2: Handling OLAF Thread Additions

If you write a new service module for the `oresat-c3` monolith, remember that it shares the GIL.

- **Avoid tight while True loops** without a `time.sleep()`.
- **Never use `threading.Lock()`** around hardware I/O. Use `queue.SimpleQueue` to pass data to a dedicated I/O thread.
- **Do not use `multiprocessing`:** Spawning sub-processes via Python's `multiprocessing` module will rapidly consume the 256MB Cgroup limit, as each child process replicates the memory footprint of the parent. If you must use multiple processes, you must recalculate the `MemoryMax` parameter.

Rule 3: Adjusting the RAM Cage (`MemoryMax`)

If a future update natively requires more than 256MB of RAM:

1. **Never enable Swap.** Leave `MemorySwapMax=0`.
2. **Calculate the Page Cache Cost:** If you increase `MemoryMax to 512M,` you are halving the eMMC flash write buffer.
3. **Re-Validate I/O:** If you shrink the page cache, you must run the dd I/O stress test. If the SD card blocks for more than 30 seconds due to the smaller buffer, the Watchdog will trigger, and your new memory limit will cause the satellite to enter a boot loop.

Rule 4: Validating Changes

If you change the architecture or update the kernel, you **must** run the validation suite located in `tools/`.

1. **Test CPU Safety:** Run `python3 stressCPU.py` alongside `measureJitter.py.` Ensure jitter doesn't devolve into pure chaos.
2. **Test OOM Recovery:** Modify `oresat-c3.service` to run `tools/memoryBomb.py`. Monitor `journalctl -u oresat-c3 -f`. You *must* see the process get killed (`status=9/KILL`) exactly at your defined `MemoryMax` limit, and automatically restart.